Assignment 3

DES Cipher (Feistel Structure)

Name: Arnav Singhal
MIS: 612203012
Final Year B. Tech. Comp. Div 1
Batch: B3

1. Introduction

The Data Encryption Standard (DES) is a classical symmetric block cipher that encrypts data in fixed 64-bit blocks using a 56-bit effective key. DES is built on a Feistel structure, which means the same round function is applied repeatedly to alternating halves of the block, and the exact same structure can be reused for decryption simply by reversing the order in which the round keys are applied. Although DES is now considered insecure for real-world use because its 56-bit key is small enough to brute-force with modern hardware, it remains the foundational example of a Feistel-network cipher.

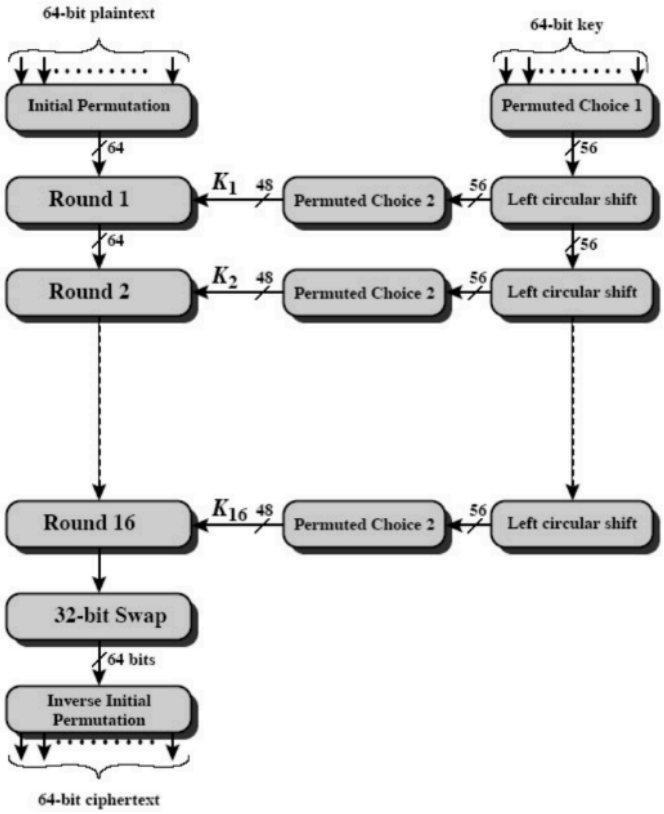


Figure 1: Overall Scheme of DES Encryption

2. The Feistel Structure

In a Feistel cipher, each 64-bit block is split into a left half L and a right half R of 32 bits each. A round function f, parameterised by a round subkey, is applied to R and XORed with L to produce the new R, while the old R becomes the new L. This is repeated for a fixed number of rounds - 16 in the case of DES. Because the round function f never needs to be inverted (only XOR and a swap are undone), the same round function can be reused unchanged for both encryption and decryption; only the order of the subkeys is reversed.

Each of the 16 DES rounds performs the following sequence of operations:

Step	Operation	Purpose
1	Split 64-bit block into L (32 bits) and R (32 bits)	Sets up the Feistel halves for this round
2	Expand R from 32 to 48 bits using table E	Matches R to the 48-bit round subkey length
3	XOR the expanded R with the round subkey	Mixes key material into the data
4	Pass the 48-bit result through 8 S-boxes	Produces 32 bits of non-linear substitution
5	Permute the 32-bit S-box output using table P	Diffuses the substituted bits
6	New R = L XOR f(R, subkey); new L = old R	Completes the Feistel swap for the round

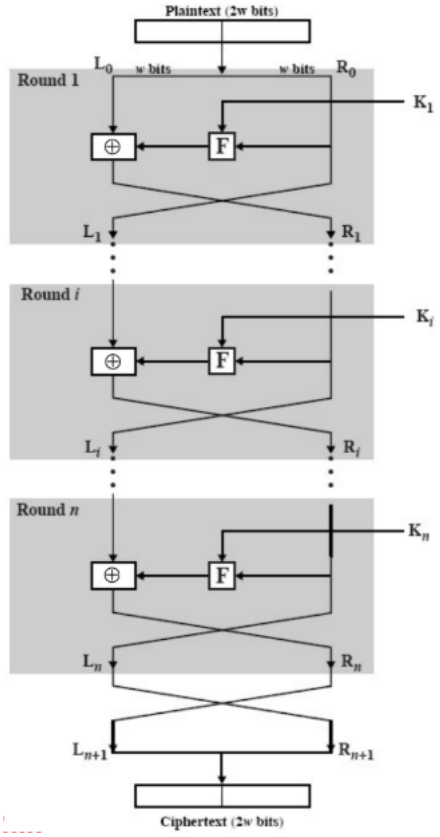


Figure 2: Feistel Cipher Structure

3. Key Schedule (56-bit Key)

DES nominally takes a 64-bit key, but 8 of those bits are parity bits and are discarded immediately, leaving an effective key length of 56 bits. From this 56-bit key, 16 separate 48-bit round subkeys (K1 through K16) are derived, one for each round:

Stage	Input size	Output size	Table / Operation Used
Initial key permutation	64 bits	56 bits	PC-1 (drops 8 parity bits)
Split into halves	56 bits	28 + 28 bits	C0 and D0
Per-round left shift	28 + 28 bits	28 + 28 bits	1 or 2 bits, per the SHIFTS schedule
Round key permutation	56 bits (Ci + Di)	48 bits	PC-2, applied 16 times to give K1..K16

The permutation choice tables PC-1 and PC-2, together with the per-round shift schedule (mostly 2-bit shifts, with 1-bit shifts on rounds 1, 2, 9, and 16), ensure that every round key depends on a different, overlapping subset of the original key bits.

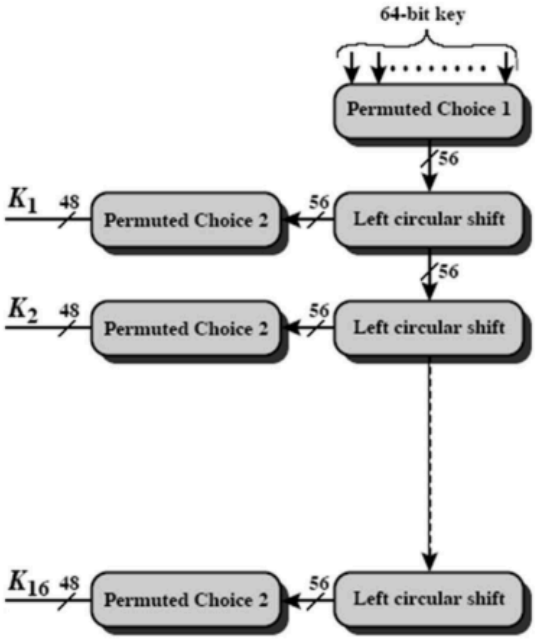


Figure 3: DES Key Generation

4. Encryption Process

Encrypting a single 64-bit block proceeds as follows:

- The block is first rearranged using the Initial Permutation (IP), which does not add cryptographic strength on its own but was originally intended to simplify loading the block into hardware registers.
- The permuted block is split into L0 and R0, and the 16 Feistel rounds described above are applied in sequence, using round keys K1 through K16 in order.
- After the 16th round, L16 and R16 are swapped once more (R16 concatenated with L16) and passed through the Final Permutation (FP), which is the exact inverse of IP.

Because encryption and decryption share the same round structure, decrypting a block uses the identical des_block routine but simply feeds the 16 round keys in reverse order (K16 down to K1).

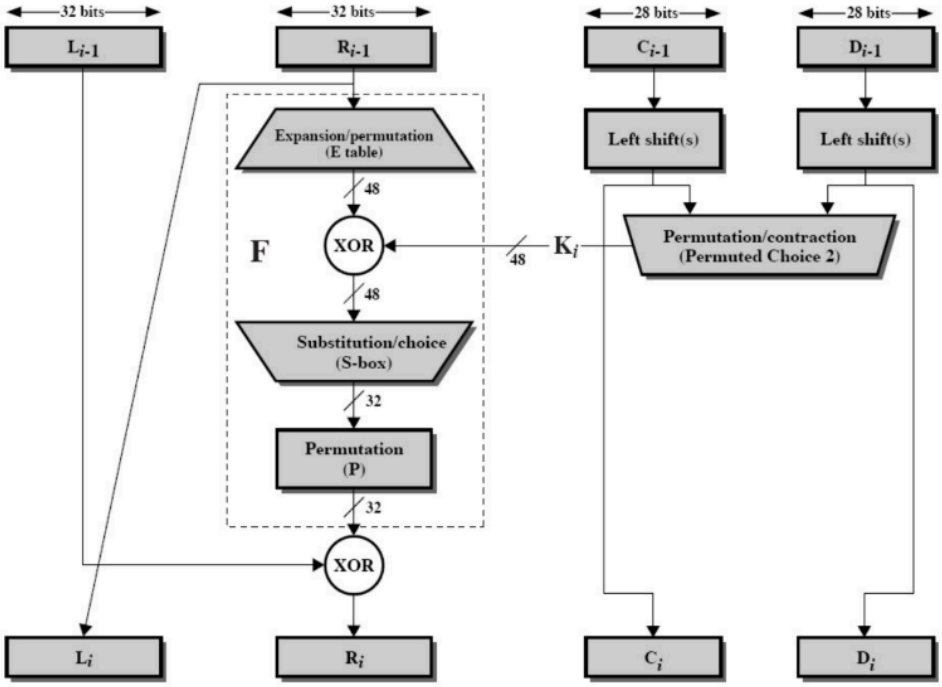


Figure 4: Single Round of DES Encryption

5. Padding

Since DES only operates on fixed 64-bit (8-byte) blocks, plaintext of arbitrary length must be padded to a multiple of 8 bytes before encryption. The implementation uses PKCS-style padding: the number of padding bytes needed is calculated, and that many bytes, each holding that same count as its value, are appended to the plaintext. On decryption, the last byte of the recovered data tells the program exactly how many padding bytes to strip off, cleanly recovering the original message regardless of its length.

6. Program Implementation

The Python program implements the complete DES algorithm from first principles: all of the standard permutation tables (IP, FP, E, P, PC-1, PC-2), all eight S-boxes, the key schedule, the Feistel round function, and block-level encryption and decryption with padding.

6.1 Core Functions

Helper functions convert between bytes and bit lists, apply a permutation table to a list of bits, XOR two equal-length bit lists, and perform the circular left shifts used while generating the round keys.

6.2 Key Schedule Generation

generate_subkeys() applies PC-1 to the raw 64-bit key, splits the result into two 28-bit halves C and D, then for each of the 16 rounds performs the scheduled left shift on both halves and applies PC-2 to the concatenated result to produce that round's 48-bit subkey.

6.3 The Feistel Round Function

feistel() expands the 32-bit right half to 48 bits via table E, XORs it with the round subkey, substitutes the result through the eight S-boxes via sbox_substitution(), and permutes the 32-bit output via table P. des_block() drives the 16 rounds (in forward or reverse key order) and applies the initial and final permutations.

6.4 Code and Output

```
IP = [58,50,42,34,26,18,10,2,
      60,52,44,36,28,20,12,4,
      62,54,46,38,30,22,14,6,
      64,56,48,40,32,24,16,8,
      57,49,41,33,25,17,9,1,
      59,51,43,35,27,19,11,3,
      61,53,45,37,29,21,13,5,
      63,55,47,39,31,23,15,7]

FP = [40,8,48,16,56,24,64,32,
      39,7,47,15,55,23,63,31,
      38,6,46,14,54,22,62,30,
      37,5,45,13,53,21,61,29,
      36,4,44,12,52,20,60,28,
      35,3,43,11,51,19,59,27,
      34,2,42,10,50,18,58,26,
      33,1,41,9,49,17,57,25]

E = [32,1,2,3,4,5,
     4,5,6,7,8,9,
```

```
    8,9,10,11,12,13,
    12,13,14,15,16,17,
    16,17,18,19,20,21,
    20,21,22,23,24,25,
    24,25,26,27,28,29,
    28,29,30,31,32,1]

P = [16,7,20,21,29,12,28,17,
    1,15,23,26,5,18,31,10,
    2,8,24,14,32,27,3,9,
    19,13,30,6,22,11,4,25]

PC1 = [57,49,41,33,25,17,9,
       1,58,50,42,34,26,18,
       10,2,59,51,43,35,27,
       19,11,3,60,52,44,36,
       63,55,47,39,31,23,15,
       7,62,54,46,38,30,22,
       14,6,61,53,45,37,29,
       21,13,5,28,20,12,4]

PC2 = [14,17,11,24,1,5,
       3,28,15,6,21,10,
       23,19,12,4,26,8,
       16,7,27,20,13,2,
       41,52,31,37,47,55,
       30,40,51,45,33,48,
       44,49,39,56,34,53,
       46,42,50,36,29,32]

SHIFTS = [1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1]

SBOX = [
[[14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7],
 [0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8],
 [4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0],
 [15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13]],

[[15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10],
 [3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5],
 [0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15],
```

```
 [13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9]],

[[10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8],
 [13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1],
 [13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7],
 [1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12]],

[[7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15],
 [13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9],
 [10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4],
 [3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14]],

[[2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9],
 [14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6],
 [4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14],
 [11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3]],

[[12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11],
 [10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8],
 [9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6],
 [4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13]],

[[4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1],
 [13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6],
 [1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2],
 [6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12]],

[[13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7],
 [1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2],
 [7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8],
 [2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11]]
]

def permute(bits, table):
    return [bits[i-1] for i in table]

def xor(a, b):
    return [x ^ y for x, y in zip(a, b)]

def left_shift(bits, n):
    return bits[n:] + bits[:n]
```

```
def bytes_to_bits(data):
    bits = []
    for byte in data:
        for i in range(7, -1, -1):
            bits.append((byte >> i) & 1)
    return bits

def bits_to_bytes(bits):
    b = bytearray()
    for i in range(0, len(bits), 8):
        byte = 0
        for bit in bits[i:i+8]:
            byte = (byte << 1) | bit
        b.append(byte)
    return bytes(b)

def generate_subkeys(key64bits):
    permuted = permute(key64bits, PC1)
    C = permuted[:28]
    D = permuted[28:]
    subkeys = []
    for shift in SHIFTS:
        C = left_shift(C, shift)
        D = left_shift(D, shift)
        combined = C + D
        subkeys.append(permute(combined, PC2))
    return subkeys

def sbox_substitution(bits48):
    output = []
    for i in range(8):
        block = bits48[i*6:(i+1)*6]
        row = (block[0] << 1) | block[5]
        col = (block[1] << 3) | (block[2] << 2) | (block[3] << 1) | block[4]
        val = SBOX[i][row][col]
        for j in range(3, -1, -1):
            output.append((val >> j) & 1)
    return output

def feistel(R, subkey):
```

```
    expanded = permute(R, E)
    xored = xor(expanded, subkey)
    substituted = sbox_substitution(xored)
    return permute(substituted, P)

def des_block(block64bits, subkeys, decrypt=False):
    permuted = permute(block64bits, IP)
    L = permuted[:32]
    R = permuted[32:]
    keys = subkeys[::-1] if decrypt else subkeys
    for key in keys:
        newL = R
        newR = xor(L, feistel(R, key))
        L, R = newL, newR
    combined = R + L
    return permute(combined, FP)

def pad(data):
    pad_len = 8 - (len(data) % 8)
    return data + bytes([pad_len] * pad_len)

def unpad(data):
    pad_len = data[-1]
    return data[:-pad_len]

def des_encrypt(plaintext, key8bytes):
    key_bits64 = bytes_to_bits(key8bytes)
    subkeys = generate_subkeys(key_bits64)
    padded = pad(plaintext)
    result = bytearray()
    for i in range(0, len(padded), 8):
        block = padded[i:i+8]
        block_bits = bytes_to_bits(block)
        cipher_bits = des_block(block_bits, subkeys, decrypt=False)
        result.extend(bits_to_bytes(cipher_bits))
    return bytes(result)

def des_decrypt(ciphertext, key8bytes):
    key_bits64 = bytes_to_bits(key8bytes)
    subkeys = generate_subkeys(key_bits64)
    result = bytearray()
```

```
    for i in range(0, len(ciphertext), 8):
        block = ciphertext[i:i+8]
        block_bits = bytes_to_bits(block)
        plain_bits = des_block(block_bits, subkeys, decrypt=True)
        result.extend(bits_to_bytes(plain_bits))
    return unpad(bytes(result))

if __name__ == "__main__":
    key = b"SECRET56"
    plaintext = input("Enter plaintext: ").encode("utf-8")
    ciphertext = des_encrypt(plaintext, key)
    decrypted = des_decrypt(ciphertext, key)
    print("Plaintext:", plaintext.decode())
    print("Key:", key.decode())
    print("Ciphertext (hex):", ciphertext.hex())
    print("Decrypted:", decrypted.decode())
```

Output:

```
arnav@arnav-HP-Laptop-15s-fq5xxx:~/Desktop/COEP/CNS/Assignment3$ python3 des.py
Enter plaintext: Cryptography and Network Security
Plaintext: Cryptography and Network Security
Key: SECRET56
Ciphertext (hex): 54e22f5d7aaed35396a7cb8df21b9bd692d7f86bd463713bab2a6df6a4c55b688cce8f03d70f4432
Decrypted: Cryptography and Network Security
arnav@arnav-HP-Laptop-15s-fq5xxx:~/Desktop/COEP/CNS/Assignment3$
```

6.5 Demonstration Result

The 33-byte plaintext is padded to 40 bytes (five 8-byte blocks) before encryption, which is why the resulting ciphertext is 80 hexadecimal characters (40 bytes) long. Decrypting the ciphertext with the same key reverses the 16 Feistel rounds in the opposite subkey order, undoes the final and initial permutations, and strips the padding bytes off the end, exactly recovering the original plaintext. This confirms that the Feistel structure, the key schedule, and the padding scheme are all implemented correctly and consistently with each other.

7. Conclusion

DES demonstrates how a Feistel network can build a full block cipher out of a single, non-invertible-in-appearance round function, by relying on XOR and a half-swap to make every round trivially reversible. Its 56-bit key schedule, expansion and permutation tables, and eight S-boxes together provide the confusion and diffusion needed for security against naive attacks, but the small key size makes it vulnerable to brute-force search on modern hardware, which is why DES was eventually superseded by Triple DES and then AES. Implementing DES from scratch, as done here, makes clear

exactly how the Feistel structure, key schedule, and round function work together to produce a complete, invertible block cipher.